

A Market Maker on a Chip

An LMSR prediction-market market maker built on an FPGA, in C++ and in Python

Dhanush Ekollu MHacks

The project in one paragraph

We built a *market maker*: a program that always stands ready to buy or sell in a market, and keeps adjusting its prices as people trade with it. We built the same market maker three times: as a digital circuit on an FPGA board, as a C++ program, and as a Python program. All three give exactly the same prices, down to the last bit, for the same sequence of trades. That lets us compare them fairly on one question: how long does each take to produce a new price, and how *consistent* is that time?

This document explains the idea from scratch. No background in trading or hardware is assumed.

Contents

1	Background: prediction markets and market makers	2
2	The pricing rule: LMSR	2
3	What we are comparing, and why	4
4	Making the math fit in hardware	5
5	The system	6
6	The FPGA design	6
7	Results	8

1 Background: prediction markets and market makers

1.1 A prediction market

A prediction market lets people trade on whether something will happen: “Will it rain in Ann Arbor on Friday?” The thing being traded is a **YES share**. A YES share pays \$1 if the event happens and nothing if it does not.

If a YES share is trading at 70¢, the market is saying the event is about 70% likely. Someone who thinks the real chance is 90% would happily buy at 70¢; someone who thinks it is 50% would happily sell. The price moves until buyers and sellers balance, so the price is a running estimate of the probability.

1.2 Why a market needs a market maker

A market only works if there is someone to trade with. If you want to buy a YES share and nobody happens to be selling at that moment, nothing happens.

A **market maker** solves this by always being available. At every moment it posts two prices:

- the **bid**: the price it will *pay* to buy a YES share from you;
- the **ask**: the price it will *charge* to sell a YES share to you.

The ask is always a little higher than the bid. The gap is called the **spread**. If the market maker buys at 49¢ and sells at 51¢, it earns 2¢ for providing the service of always being there.

1.3 The market maker’s problem

Suppose many people buy YES from the market maker and few sell. It has now sold a lot of YES shares. If the event happens, it owes \$1 on every one of them. This pile-up is called **inventory**, and it is the market maker’s main risk.

A sensible market maker reacts by *raising its prices* as it sells more. Higher prices discourage further buying and encourage people to sell back, which brings it back toward balance. Higher prices also reflect what the buying is telling it: if everyone wants YES, YES is probably more likely than the old price said.

So a market maker needs a rule that turns “how much have I sold?” into “what should my bid and ask be now?” Every time a trade happens it applies the rule again and posts new prices. Doing this quickly matters: while its prices are out of date, anyone who has noticed can trade against the stale price.

2 The pricing rule: LMSR

We use a well-known rule called **LMSR** (logarithmic market scoring rule), designed by the economist Robin Hanson for prediction markets. Its appeal is that the market maker’s whole memory is *one number*.

2.1 One number of state

Let d be the net number of YES shares the market maker has sold (sales minus buy-backs). The fair price of a YES share is

$$p = \frac{1}{1 + e^{-d/b}}.$$

This S-shaped curve is shown in Figure 1. When $d = 0$ the price is 50¢. As the market maker sells more YES (d grows) the price climbs toward \$1, and as it buys YES back (d falls) the price drops toward 0.

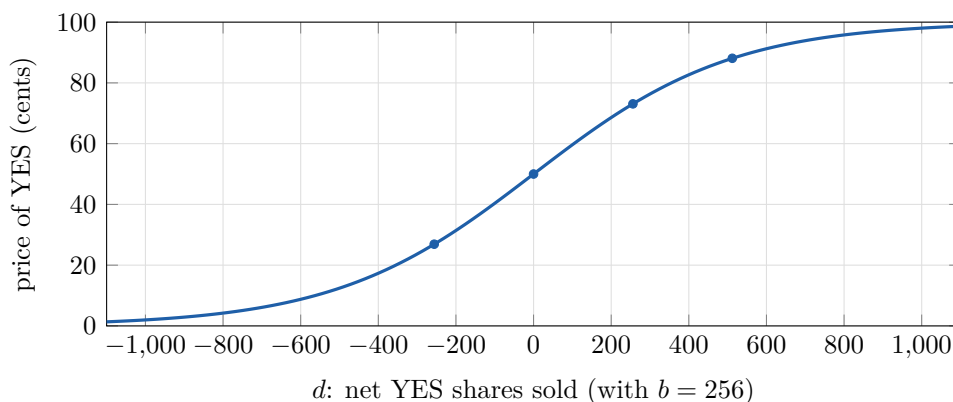


Figure 1: The LMSR price curve. Every 256 shares sold (one “ b ”) moves the price a fixed step along this curve: 50¢ $\rightarrow$ 73¢ $\rightarrow$ 88¢.

2.2 The liquidity setting b

The number b controls how quickly the price reacts. With a small b , a few trades move the price a lot. With a large b , it takes many trades. It also sets the most the market maker can ever lose, which is $b \ln 2$ dollars no matter how anyone trades. For $b = 256$ that is about \$177. This cap is the second big appeal of LMSR: the risk is known in advance.

2.3 From the curve to a bid and an ask

The market maker quotes for a fixed number of shares at a time, the **quote size** s (8 shares by default). LMSR defines a cost function

$$C(d) = b \ln(1 + e^{d/b}),$$

and the price of a trade is the change in C . So:

$$\begin{aligned} \text{ask} &= \frac{C(d+s) - C(d)}{s} && \text{(average price to sell } s \text{ more shares)} \\ \text{bid} &= \frac{C(d) - C(d-s)}{s} && \text{(average price to buy } s \text{ shares back)} \end{aligned}$$

The ask comes out slightly above the fair price and the bid slightly below, so a spread appears naturally. We then round the ask *up* and the bid *down* to whole cents, so rounding never costs the market maker money. Table 1 shows what our market maker actually quotes.

YES shares sold (d)	fair price	bid	ask
-256	26.9¢	26¢	28¢
0	50.0¢	49¢	51¢
128	62.2¢	61¢	63¢
256	73.1¢	72¢	74¢
512	88.1¢	87¢	89¢
1024	98.2¢	98¢	99¢

Table 1: Quotes produced by our market maker ($b = 256$, quote size 8).

2.4 The whole algorithm

1. Post a bid and an ask computed from d .
2. A trader buys q shares at the ask: set $d \leftarrow d + q$.
A trader sells q shares at the bid: set $d \leftarrow d - q$.
3. Recompute the bid and ask from the new d and post them. Repeat.

Step 3 is the part we time. We call it **tick-to-quote**: the time from learning about a trade to having the new prices ready.

3 What we are comparing, and why

3.1 Why put this on an FPGA?

An FPGA is a chip whose internal wiring can be reconfigured. Instead of running a program one instruction at a time, you describe a circuit, and the chip *becomes* that circuit. Trading firms use FPGAs for two reasons:

- **No operating system in the way.** A circuit reacts to its input directly. A program must wait for the operating system to hand it the data.
- **Perfectly repeatable timing.** A circuit takes the same number of clock ticks every time. A program's timing varies: another program interrupts it, or the data it needs is not in the processor's fast memory. This variation is called **jitter**.

3.2 An honest expectation

We are not claiming the FPGA is faster at the arithmetic. Our pricing step is tiny, and a laptop processor runs billions of operations per second, while our hobby-grade FPGA board runs at 48 million clock ticks per second. On the *typical* request we expect the C++ program to win.

The FPGA’s advantage is consistency. In trading, the slow cases matter most: the one request in a thousand that takes far longer than usual is exactly when a market maker’s prices are out of date. A design whose worst case equals its typical case has no such moments. That is the result this project is built to show.

3.3 Keeping the comparison fair

- **Identical answers.** All three versions follow one written specification and must agree bit for bit, so none can be faster by doing less work.
- **Identical input.** One fixed file of trades is replayed into all three.
- **Compute time only.** Each version measures itself from “the whole request has arrived” to “the new prices are ready”, and reports that number inside its reply. The time spent moving bytes between the laptop and the market maker is left out for all three.

4 Making the math fit in hardware

The formulas above use logarithms, exponentials, multiplication and division. Our FPGA has none of these built in, not even a multiplier. Each design decision below removes one obstacle.

Decision	Why
Whole numbers only, no decimals	Decimal (floating-point) arithmetic can round differently in different languages. Whole numbers behave identically everywhere, so the three versions can match exactly. Prices carry 16 extra binary digits of precision internally.
A lookup table for the hard function	$\ln(1 + e^x)$ is too expensive to compute in a small circuit. It equals a straight line plus a small bump that fades to zero, so we store only the bump: 2,048 precomputed values. Looking up a value replaces computing it.
Powers of two for b and s	b is 64, 128 or 256, and s is a power of two no larger than b . Multiplying or dividing by a power of two is just shifting digits, which costs almost nothing in hardware. It also lets one table serve every setting.
Round in the market maker’s favour	Ask rounds up, bid rounds down, to whole cents. Before rounding, the whole-number price is within 0.004¢ of the exact formula.
Stop quoting instead of forcing a price	If the ask would exceed 99¢ (or the bid fall below 1¢), that side of the quote is withdrawn. Forcing the price back into range would mean trading at a price worse than the rule allows.

With these choices, one new quote costs three table lookups, two subtractions, and some shifting and rounding. Nothing else.

5 The system

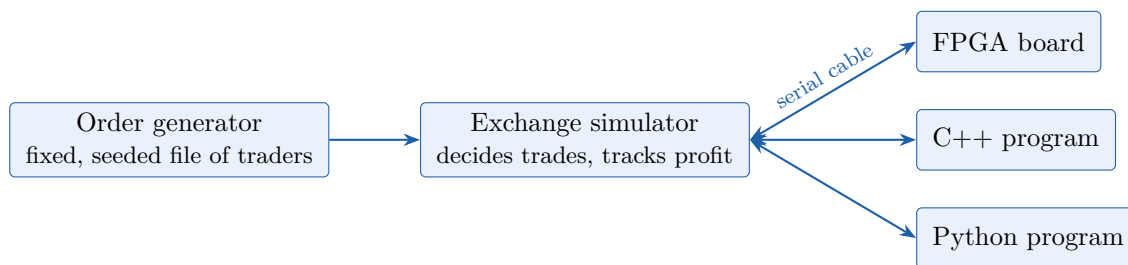


Figure 2: The same trades are replayed into each of the three market makers.

5.1 Simulated traders

The order generator writes a file of trader arrivals from a random seed, so the same seed always gives the same file. There is a hidden “true” probability that the market maker cannot see. Two kinds of trader arrive:

- **Noise traders** buy or sell at whatever the current price is. They are the market maker’s source of income, because they pay the spread.
- **Informed traders** know the true probability. They buy only when the ask is below it and sell only when the bid is above it. They cost the market maker money, but they are also what pulls its price toward the truth.

The true probability can jump partway through (“news”), and we watch the market maker’s price follow it.

5.2 The exchange simulator

The exchange reads the file one trader at a time, looks at the market maker’s current bid and ask, decides whether that trader trades, tells the market maker, and reads back the new prices. It keeps the books: the market maker’s inventory, its cash, and its profit or loss.

5.3 The messages

The exchange and a market maker talk in short fixed-size messages. A request is 5 bytes (for example “a trader bought 8 shares”). A reply is 13 bytes: the new bid and ask, the inventory, a running count of trades, and how long the computation took. Each message ends with a checksum so a corrupted one is detected and ignored.

6 The FPGA design

The board is a “HailBoard” with a Lattice iCE40 HX4K FPGA: a small, inexpensive chip with no multipliers and about 16 kilobytes of on-chip memory. It talks to the laptop over a serial link at

115,200 bits per second. The design is written in Verilog, a language for describing circuits.

The board supplies a 12 MHz clock. An on-chip clock multiplier (a PLL) turns that into 48 MHz, so the circuit takes 48 million steps per second and each step lasts about 21 nanoseconds.

6.1 The pipeline

The circuit is a chain of small modules. Bytes arrive from the laptop, become a request, get priced, and leave as a reply.

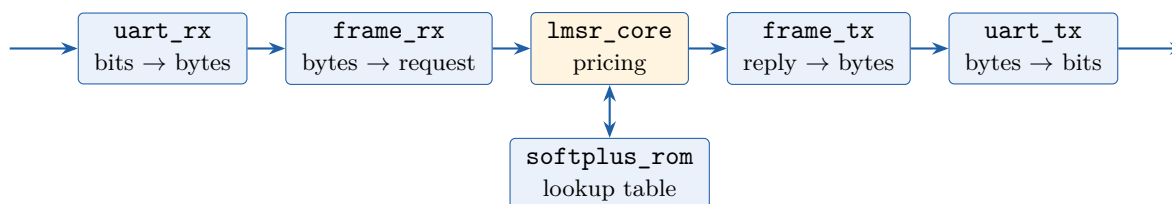


Figure 3: The modules of the FPGA market maker.

Module	What it does
uart_rx	Reads the serial wire and assembles bits into bytes. It finds the start of each byte and then samples the middle of every bit, where the signal is most stable.
frame_rx	Collects 5 bytes into a request and checks the checksum. If a message stalls halfway, it gives up after 10 milliseconds so one lost byte cannot jam the link.
softplus_rom	The lookup table, held in the chip's block memory.
lmsr_core	The market maker itself: it holds d , the settings and the current prices, and computes new prices after each request.
frame_tx	Packs the reply into 13 bytes and hands them out one at a time.
uart_tx	Sends each byte down the serial wire, bit by bit.
top	Connects everything, reads the buttons, drives the displays, and counts how many clock ticks each request takes.

6.2 Inside the pricing core: eight clock ticks

A digital circuit advances in steps driven by a clock. The pricing core handles every request in exactly eight ticks:

1. Apply the request: update d for a trade, or change a setting.
2. Ask the table for the value at $d - s$.
3. Ask for the value at d . The first answer arrives and is saved.
4. Ask for the value at $d + s$. The second answer arrives.

5. The third answer arrives.
6. Subtract: the upper difference gives the ask, the lower gives the bid.
7. Shift (the multiply by b/s) and round to whole cents.
8. Add any extra spread and decide whether each side should be withdrawn.

The three lookups happen on separate ticks because the chip's memory answers one question per tick. The work is split into small steps so that each one finishes comfortably inside a 21-nanosecond tick; that is what allows the fast clock.

One more tick is spent checking the incoming message, giving **9 ticks in total, or 188 nanoseconds**, for every request without exception. There is no branch in the circuit that could make one request take longer than another.

6.3 Board controls

The eight-digit display shows the bid, the ask, and the number of trades so far, in decimal; a withdrawn price shows as dashes. Four push buttons control a live demonstration:

Button	Action
KEY0	Restart the market maker and replay the trades from the beginning
KEY1	Kill switch: withdraw all prices immediately (press again to resume quoting)
KEY3	Pause the stream of trades at the current one
KEY2	Resume the stream

A **kill switch** is a standard safety feature of real trading systems: one control that stops all quoting at once if something looks wrong, without erasing the current position. The trades themselves are stored on the laptop, so the restart, pause and resume buttons work by sending a short message to the laptop program that feeds the board.

The slide switches can take over the settings: with switch 17 up, other switches choose b , the quote size and the extra spread, and the prices on the display change the moment a switch moves.

7 Results

Everything in this section has been measured. Nothing is estimated.

7.1 Correctness

- **Accuracy.** Across all 60,392 combinations of settings and inventory, the whole-number price differs from the exact formula by at most 0.0038¢ before rounding to cents.
- **Identical behaviour.** A set of 6,971 test requests, chosen to exercise every rule (price limits, bad sizes, the kill switch, every setting), produces identical replies from the FPGA board, the C++ program and the Python program.

- **Identical trading runs.** Four full simulated markets, between 1,290 and 3,297 trades each, produce byte-for-byte identical streams of prices from all three.
- **Size.** The design uses 2,011 of the chip’s 7,680 logic cells and 12 of its 32 memory blocks. The tools estimate it could run at about 59 MHz, comfortably above the 48 MHz it needs.

The FPGA design was first checked in simulation, where a test program plays the role of the laptop and compares every reply against the Python reference, and only then loaded onto the board and checked again.

7.2 Tick-to-quote latency

Table 2 is the main result. “p99” means 99% of replies were at least this fast; “max” is the single slowest reply. The last column is the slowest reply divided by the typical one: a measure of jitter.

Version	Median	p99	p99.9	Max	Max ÷ median
FPGA	188 ns	188 ns	188 ns	188 ns	1×
C++	41 ns	166 ns	291 ns	1,042 ns	25×
Python	2,500 ns	7,375 ns	11,291 ns	14,458 ns	6×

Table 2: Compute time per reply, from one benchmark run: the same 4,000 trader arrivals replayed into each version at 400 requests per second, giving 3,262 replies each. All three produced identical prices. The FPGA took 9 clock ticks at 48 MHz on every reply.

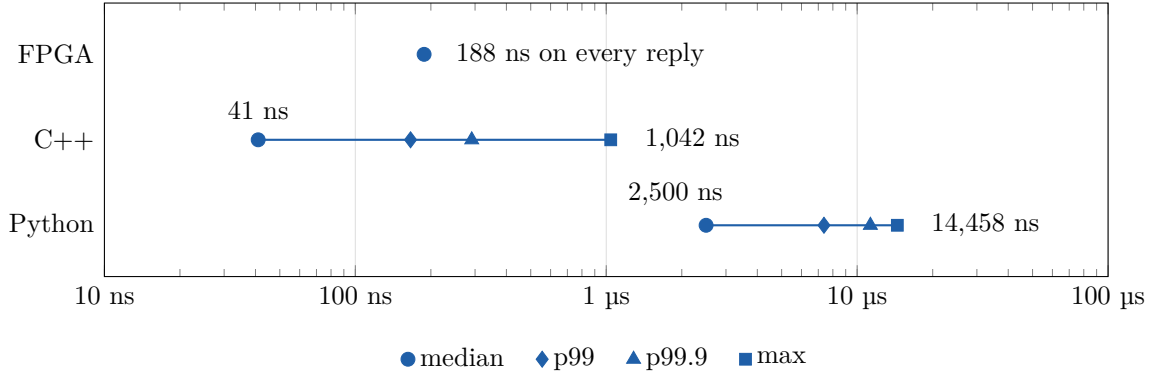


Figure 4: The same numbers on a logarithmic axis. Each software version is a line from its typical reply to its slowest. The FPGA is a single point.

How to read it:

- **C++ is fastest on a typical request**, about 4.6 times faster than the FPGA. (41 ns is one tick of the laptop’s timer, so the true figure is at or below that.) This is what we expected.
- **C++ is not fastest on its worst request.** Its slowest reply took 1,042 ns, about 5.5 times slower than the FPGA’s.
- **The FPGA has no spread at all.** Typical and worst case are the same 9 ticks.

- **Python** is slower than the FPGA throughout, by about 13 times on a typical request and 77 times on its slowest.

The software numbers were taken on an otherwise idle laptop, and they move from run to run: an earlier run of the same benchmark had a slowest C++ reply of 1,542 ns and a slowest Python reply of 29,334 ns. The FPGA figure does not move. A busy machine would lengthen the software tails and leave the FPGA unchanged; we did not measure that.

7.3 Choosing the fee

The market maker can add an extra spread on top of what LMSR gives. A wider spread earns more per trade but drives customers away. To find the balance we gave noise traders a private limit price (within 10¢ of the true probability) and ran every combination of b and extra spread over 20 different simulated markets of 4,000 trader arrivals each.

Extra spread per side	$b = 64$	$b = 128$	$b = 256$
0¢	\$44 (1,337)	\$28 (1,394)	\$5 (1,415)
2¢	\$112 (916)	\$106 (999)	\$93 (1,061)
4¢	\$124 (602)	\$133 (704)	\$131 (776)
6¢	\$95 (332)	\$116 (432)	\$123 (507)
8¢	\$10 (41)	\$47 (154)	\$70 (244)

Table 3: Average profit, with the average number of trades in brackets, when 30% of traders are informed. Full results for 10%, 30% and 50% informed are in `docs/fee_tuning.md`.

Profit peaks at about 4¢ of extra spread per side with $b = 128$, and the same setting is best at all three informed-trader levels we tried. With no extra spread the market maker barely breaks even; beyond 6¢ it has priced itself out of the market.

7.4 More than two outcomes

Real prediction markets often have several outcomes (“who wins the election?”). As a software prototype we extended the same arithmetic to N outcomes: the general LMSR formula can be built up two outcomes at a time, and each step uses the same kind of lookup table as the two-outcome design, plus one small multiplication. For two outcomes it reproduces the main design’s quotes exactly (2,201 of 2,201 compared). For up to eight outcomes its prices stay within 0.07¢ of the exact formula. This version runs in Python only; it has not been built on the FPGA.

What comes next

- **Live market data.** The market maker can already take a reference price from a real Polymarket market and quote around it without placing real orders. This path has been unit-tested but not yet run end to end on the board.
- **Latency under load.** Repeat the benchmark with the laptop busy, to show the software tails growing while the FPGA stays fixed.